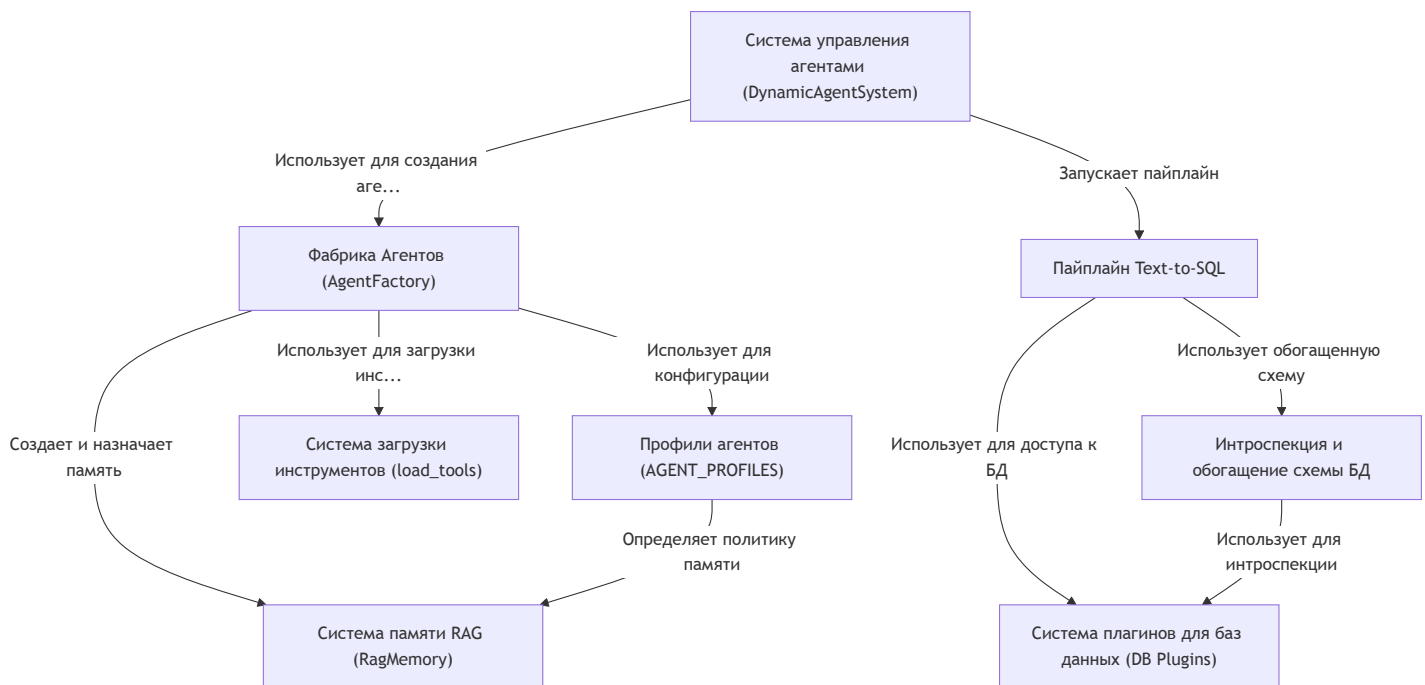


Описание проекта MultiAgent

Этот проект представляет собой *продвинутую мультиагентную систему*, где **главный дирижер (Система управления агентами)** координирует команду специализированных ИИ-агентов для выполнения сложных задач. Система использует **Фабрику Агентов** для динамического создания специалистов на основе готовых "*должностных инструкций*" (*Профили агентов*), оснащая их необходимыми инструментами. Ключевой особенностью является **умная RAG-память**, позволяющая агентам учиться на прошлом опыте. Одной из самых мощных функций является **пайплайн Text-to-SQL**, который преобразует человеческий язык в запросы к базам данных, поддерживая различные СУБД через **систему плагинов** и автоматически **обогащая схемы данных** для лучшего понимания.



Глаvas

1. Система управления агентами (DynamicAgentSystem)
2. Фабрика Агентов (AgentFactory)
3. Профили агентов (AGENT_PROFILES)
4. Система памяти RAG (RagMemory)
5. Система загрузки инструментов (load_tools)
6. Пайплайн Text-to-SQL
7. Система плагинов для баз данных (DB Plugins)

Глава 1: Система управления агентами (DynamicAgentSystem)

Добро пожаловать в мир MultiAgent ! В этом курсе мы разберем, как построить сложную систему, где множество интеллектуальных "агентов" работают вместе для решения комплексных задач. И начнем мы с самого сердца нашей системы.

Представьте, что вам нужно построить дом. Вы же не будете сами класть кирпичи, проводить электричество и проектировать водопровод? Скорее всего, вы наймете руководителя проекта. Он поймет вашу задачу, найдет нужных специалистов (каменщика, электрика, сантехника), поставит им задачи и проконтролирует, чтобы всё было сделано качественно и в срок.

В нашем проекте роль такого руководителя выполняет **DynamicAgentSystem**.

Что такое DynamicAgentSystem ?

DynamicAgentSystem — это главный "дирижер" или "менеджер проекта" всей нашей системы. Он получает от вас общую, высокоуровневую задачу (например, "проанализируй последние тенденции в области искусственного интеллекта"), а затем организует всю работу для её выполнения.

Вот его ключевые обязанности:

1. **Анализ Задачи:** Он изучает ваш запрос, чтобы понять, какие "профессиональные навыки" потребуются для ответа.
2. **Формирование Команды:** Определив нужные навыки, он обращается к [Фабрике Агентов \(AgentFactory\)](#), чтобы "нанять" команду агентов-специалистов. Например, ему могут понадобиться `researcher` (исследователь) для поиска информации и `analyst` (аналитик) для её структурирования.
3. **Координация Работы:** Он передает задачу главному в команде — агенту-менеджеру, который уже делегирует подзадачи специалистам.
4. **Сбор Результатов:** После того как все агенты выполнили свою работу, DynamicAgentSystem собирает все результаты в единый, понятный отчет и предоставляет

его вам.

Проще говоря, `DynamicAgentSystem` — это та точка входа, с которой вы взаимодействуете. Вы даете ему задачу, а он возвращает вам готовое решение.

Как это использовать?

Работать с `DynamicAgentSystem` очень просто. Весь сложный процесс скрыт "под капотом". Взгляните на файл `main.py` — это всё, что нужно для запуска системы.

```
// main.py

import asyncio
from agent_system import DynamicAgentSystem

async def main():
    # 1. Создаем экземпляр нашей системы
    system = DynamicAgentSystem()

    # 2. Формулируем нашу сложную задачу
    complex_task = "Почему компании массово отказываются от Agile методологии?"

    # 3. Запускаем координацию и ждем результат
    content = await system.coordinate(complex_task)

    print(content)
```

Давайте разберем этот код по шагам:

1. `system = DynamicAgentSystem()` : Мы создаем нашего "менеджера проекта". Он готов к работе.
2. `complex_task = "..."` : Мы определяем задачу, которую хотим решить. Это может быть любой сложный вопрос.
3. `await system.coordinate(complex_task)` : Это самая важная строка. Мы передаем нашу задачу системе и просим её начать "координировать" выполнение.

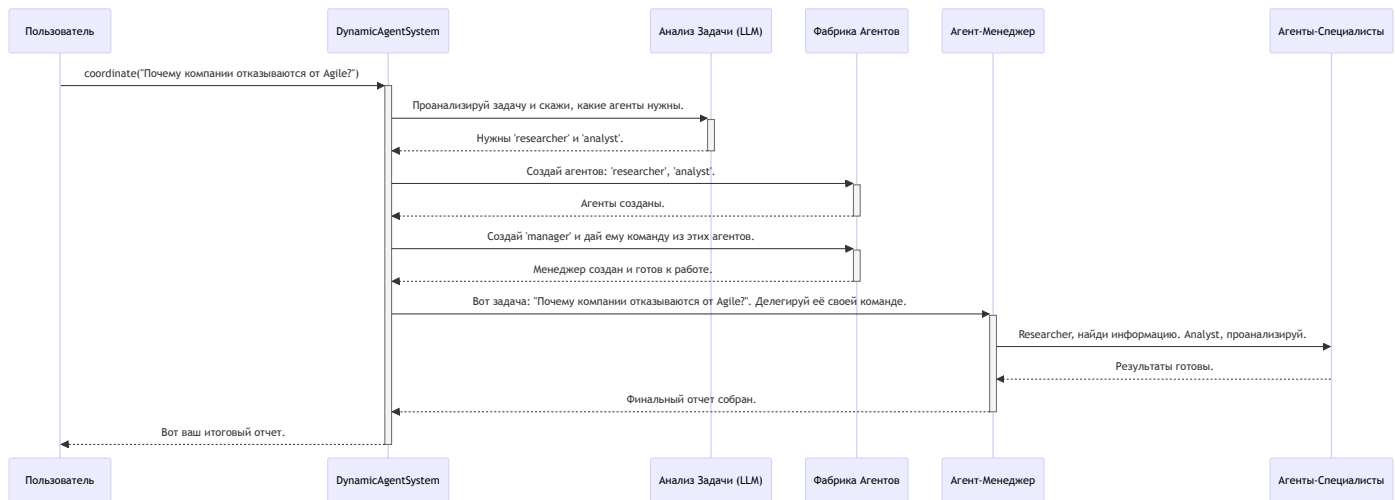
Что произойдет после запуска?

Система оживет! `DynamicAgentSystem` проанализирует задачу, создаст необходимых агентов (например, "Исследователя" для поиска статей об Agile и "Аналитика" для выявления причин),

организует их работу и в итоге выведет в консоль подробный отчет, отвечающий на ваш вопрос.

Как это работает "под капотом"?

Давайте заглянем внутрь `DynamicAgentSystem` и посмотрим, что происходит после вызова метода `coordinate()`. Весь процесс можно представить в виде простой диаграммы.



Теперь рассмотрим ключевые фрагменты кода из файла `agent_system.py`, которые реализуют эту логику.

Шаг 1: Анализ задачи

Все начинается с анализа вашей задачи. `DynamicAgentSystem` использует для этого мощную языковую модель (LLM).

```
// agent_system.py -> метод coordinate()

async def coordinate(self, initial_task: str, ...):
    # ...
    # Анализируем задачу, чтобы понять, какие агенты нам нужны.
    agent_types, pipeline_type = await self.analyze_task(initial_task)
    # ...
```

Метод `analyze_task` отправляет специальный запрос к LLM, который возвращает список типов агентов (например, `['researcher', 'analyst', 'manager']`), идеально подходящих для решения именно этой задачи.

Шаг 2: Создание команды агентов

Получив список необходимых специалистов, `DynamicAgentSystem` использует [Фабрику Агентов \(AgentFactory\)](#) для их создания.

```
// agent_system.py -> метод coordinate()

# Создаем агентов-специалистов по списку
for agent_type in agent_types:
    if agent_type != 'manager':
        # Фабрика создает и настраивает каждого агента
        agent = self.factory.create_agent(agent_type, ...)
        self.agent_pool[agent.name] = {'agent': agent}
```

Здесь система в цикле проходит по списку `agent_types` и просит фабрику создать каждого агента.

Шаг 3: Назначение и запуск менеджера

Команда не может работать без руководителя. Поэтому, когда все специалисты "наняты", система создает главного — `manager-agent`.

```
// agent_system.py -> метод coordinate()

# Создаем менеджера последним, когда его команда уже собрана
manager_agent = self.factory.create_agent('manager', ...)

# Запускаем менеджер-агента, передавая ему основную задачу
answer = manager_agent.run(manager_instructions)
```

Обратите внимание: менеджер создается **последним**. Это важно, потому что при создании ему передается уже готовая команда агентов-специалистов. После этого метод `run()` запускает весь процесс делегирования и решения задачи.

Шаг 4: Формирование итогового отчета

Когда `manager_agent` закончит свою работу и вернет `answer`, `DynamicAgentSystem` собирает всю информацию воедино и готовит финальный отчет для вас.

```
// agent_system.py -> метод coordinate()

# ...
# Модифицируем формирование отчета
report = []
report.append("=== ИТОГОВЫЙ ОТЧЕТ ===\n")
report.append(f"🔍 Исходная задача: {initial_task}")

# Добавляем подробный ответ от менеджера
report.append("💡 Ответ менеджера:")
report.append(f"Подробный отчет:\n{answer}")

return "\n".join(report)
```

Этот код собирает все части головоломки вместе: исходную задачу, результаты работы каждого агента (здесь для простоты показан только ответ менеджера) и представляет их в удобном для чтения виде.

Заключение

В этой главе мы познакомились с `DynamicAgentSystem` — мозговым центром и главным дирижером нашего проекта. Мы узнали, что он:

- Выполняет роль "менеджера проекта".
- Анализирует задачу и определяет, какие специалисты нужны.
- Динамически создает команду агентов.
- Координирует их работу для достижения цели.
- Собирает итоговый отчет.

Теперь, когда мы понимаем, кто управляет всем оркестром, самое время узнать, как создаются сами "музыканты". В следующей главе мы подробно разберем, как работает "отдел кадров" нашей системы.

Перейдем к изучению [Главы 2: Фабрика Агентов \(AgentFactory\)](#).

Глава 2: Фабрика Агентов (AgentFactory)

В [предыдущей главе](#) мы познакомились с `DynamicAgentSystem` — главным "менеджером проекта", который координирует всю работу. Мы узнали, что когда ему поступает задача, он определяет, какие специалисты нужны, и просит их "нанять". Но кто же занимается этим наймом? Кто собирает агентов по частям, как конструктор?

За эту задачу отвечает `AgentFactory`, или "Фабрика Агентов". Если `DynamicAgentSystem` — это руководитель проекта, то `AgentFactory` — это его отдел кадров, который умеет быстро находить и подготавливать нужных сотрудников.

Что такое `AgentFactory` ?

Представьте, что вы собираете команду для съемок фильма. Вам нужен режиссер, оператор, сценарист. У каждого из них своя роль, свои инструменты (камера, печатная машинка) и свои инструкции. `AgentFactory` — это кастинг-директор и менеджер по оборудованию в одном лице.

Её основная задача — автоматизировать создание и настройку агентов. Вместо того чтобы каждый раз вручную писать код для нового агента, мы просто говорим фабрике: "Мне нужен агент типа `researcher`", и она делает всё остальное:

1. **Читает "профиль"**: Фабрика находит "личное дело" (конфигурационный файл) для `researcher`. В нем описано всё: его должностные обязанности, какими инструментами он должен владеть и какой у него уровень интеллекта (какую модель ИИ использовать).
2. **Выдает "инструменты"**: Согласно профилю, фабрика подключает к агенту необходимые инструменты, например, доступ к поисковой системе.
3. **Формирует "инструкции"**: Она создает для агента подробную инструкцию (промпт), объясняя его роль и цели.
4. **Подключает "память"**: Каждому агенту выделяется персональная память, чтобы он мог учиться на основе предыдущих действий.
5. **"Собирает" агента**: Наконец, она объединяет всё это в единый, готовый к работе объект-агент.

Такой подход делает систему невероятно гибкой. Нужно добавить нового специалиста? Просто создайте для него новый файл-профиль, и фабрика научится его "собирать"!

Как это работает?

Напрямую с `AgentFactory` вы, как пользователь, почти не взаимодействуете. Эту работу за вас делает `DynamicAgentSystem`. Но важно понимать, как происходит этот процесс.

Вот как `DynamicAgentSystem` использует фабрику для создания агента-исследователя:

```
// Этот код находится внутри DynamicAgentSystem
// Предположим, у нас уже есть экземпляр фабрики
self.factory = AgentFactory()

# ...

# Просим фабрику создать агента типа 'researcher' для текущей сессии
researcher_agent = self.factory.create_agent(
    profile_type='researcher',
    session_id='session_123',
    task='Найти информацию о методологии Agile'
)
```

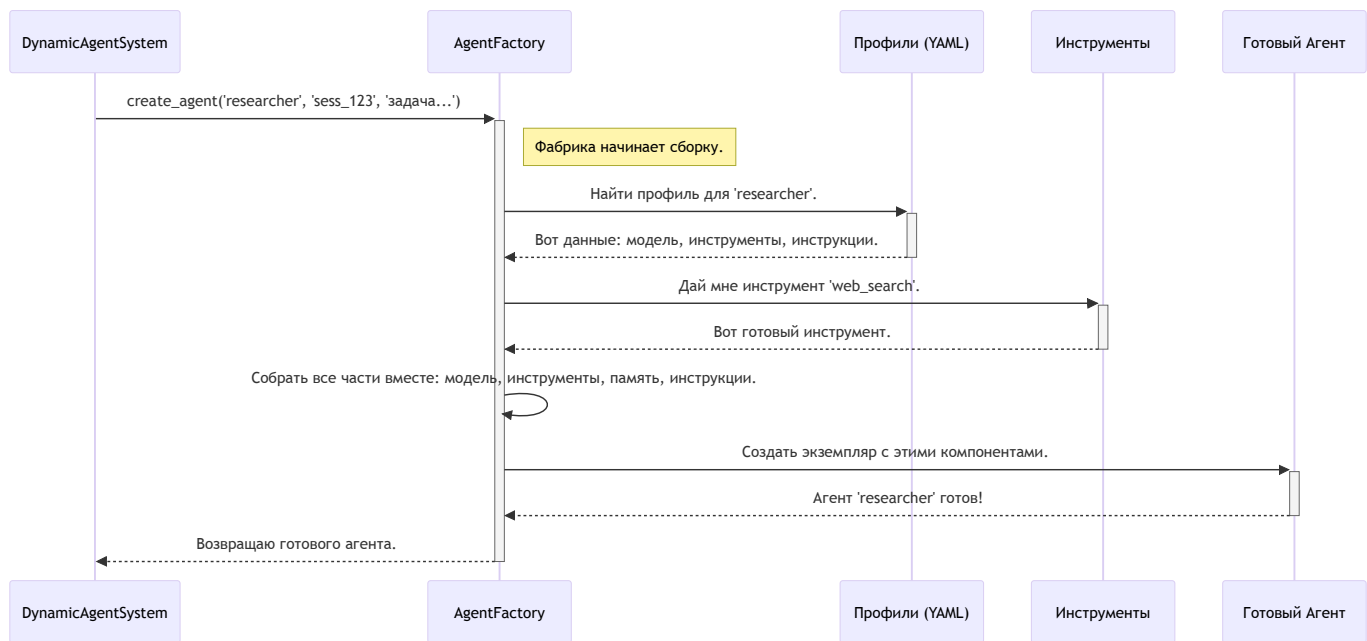
Разберем параметры:

- `profile_type='researcher'` : Мы говорим фабрике, какой "чертеж" использовать. Это ключ, по которому она найдет нужный [профиль агента](#).
- `session_id='session_123'` : Уникальный идентификатор текущей задачи. Он нужен, чтобы память у агентов из разных задач не перемешивалась.
- `task='...'` : Сама задача, которая поможет фабрике лучше настроить инструкции для агента.

В результате `researcher_agent` — это полностью сконфигурированный и готовый к работе объект.

Что происходит "под капотом"?

Процесс создания агента можно сравнить со сборочной линией на заводе. Давайте посмотрим на основные этапы этого конвейера.



Теперь заглянем в код файла `agent_factory.py` , чтобы увидеть, как это реализовано.

Шаг 1: Найти "профиль" агента

Все начинается с метода `create_agent` . Первым делом он ищет нужный профиль в глобальном словаре `AGENT_PROFILES` .

```
# agent_factory.py -> метод create_agent()

def create_agent(self, profile_type: str, ...):

    if profile_type not in AGENT_PROFILES:
        raise ValueError(f"Неизвестный тип профиля: {profile_type}")

    # Загружаем "чертеж" из словаря AGENT_PROFILES
    profile = AGENT_PROFILES[profile_type]

    # ... дальнейшая сборка ...
```

`AGENT_PROFILES` — это специальная переменная, которая загружает все `.yaml` файлы из папки `agent_profiles` . Каждый файл — это "чертеж" для одного типа агента. Подробнее мы разберем это в [следующей главе](#).

Шаг 2: Подготовить инструменты

В профиле указан список инструментов, которые нужны агенту. Фабрика берет этот список и "выдает" агенту каждый инструмент.

```
# agent_factory.py -> метод create_agent()

# Получаем список названий инструментов из профиля
profile_tools = profile.get('tools', [])

# Создаем реальные объекты инструментов для каждого названия
tools = [self._create_tool(tool_name) for tool_name in profile_tools]
```

Метод `_create_tool` находит нужный инструмент в общем реестре. Это позволяет нам централизованно управлять всеми доступными в системе инструментами, о чем мы поговорим в главе [Система загрузки инструментов \(load_tools\)](#).

Шаг 3: Настроить память

Каждому агенту нужна своя память, чтобы он не забывал, что делал на предыдущих шагах. Фабрика создает персональный экземпляр памяти для каждого.

```
# agent_factory.py -> метод create_agent()

# Создаем RAG-память
rag_memory = create_rag_memory(
    session_id=session_id,
    agent_name=agent_id,
    profile_config=profile # Передаем профиль для настройки политик памяти
)
```

Здесь мы используем `create_rag_memory` для создания умной памяти, которая умеет не только хранить, но и находить релевантную информацию. Подробнее об этом в главе [Система памяти RAG \(RagMemory\)](#).

Шаг 4: Собрать финальный экземпляр агента

Когда все компоненты готовы — модель ИИ, инструменты, память и инструкции — фабрика собирает их вместе, создавая объект агента (`CodeAgent` или `ToolCallingAgent`).

```
# agent_factory.py -> метод create_agent()

# Создаем композитный промпт (инструкции)
composite_prompt = _build_composite_prompt(profile, ...)

# Финальная сборка! Создаем экземпляр агента
agent = CodeAgent(
    tools=tools,
    model=profile.get('model'),
    instructions=composite_prompt,
    # ... другие параметры ...
)

# Заменяем стандартную память на нашу RAG-память
agent.memory = rag_memory
```

Этот код — кульминация всего процесса. Он как будто защелкивает последнюю деталь конструктора. Теперь `agent` — это полностью автономная единица, готовая к выполнению задач.

Заключение

В этой главе мы разобрали `AgentFactory` — один из самых важных механизмов, обеспечивающих гибкость и масштабируемость нашего проекта. Мы узнали, что фабрика:

- Действует как "отдел кадров", который автоматически собирает агентов.
- Использует "профили" как чертежи для сборки.
- Оснащает агентов необходимыми инструментами, моделью ИИ, памятью и инструкциями.
- Позволяет легко добавлять в систему новых агентов-специалистов, не меняя основной код.

Теперь, когда мы понимаем, как агенты создаются, самое время подробно изучить те самые "чертежи", по которым их собирают.

Переходим к [Главе 3: Профили агентов \(AGENT_PROFILES\)](#).

Глава 3: Профили агентов (AGENT_PROFILES)

В [предыдущей главе](#) мы узнали, как AgentFactory работает в качестве "отдела кадров", собирая и настраивая для нас агентов. Мы упомянули, что фабрика использует специальные "профили" или "чертежи" для каждого типа агента. Но что это за чертежи и как они выглядят?

Пришло время заглянуть в папку с "личными делами" наших агентов и познакомиться с AGENT_PROFILES — конфигурационными файлами, которые делают нашу систему такой гибкой и простой в настройке.

Что такое профиль агента и зачем он нужен?

Представьте, что вы нанимаете нового сотрудника. Вам нужна **должностная инструкция**, в которой четко прописано:

- **Должность:** "Аналитик данных".
- **Обязанности:** "Собирать данные, строить отчеты, находить закономерности".
- **Необходимые инструменты:** "Доступ к базе данных, лицензия на Tableau, Python".
- **Квалификация:** "Должен уметь работать с большими моделями ИИ".

Профили агентов — это и есть такие должностные инструкции для наших цифровых сотрудников. Они хранятся в простых текстовых файлах формата YAML и описывают всё, что нужно AgentFactory для создания полностью готового к работе агента.

Главное преимущество такого подхода — **гибкость**. Вам нужен новый тип агента? Например, переводчик? Вам не нужно лезть в сложный код AgentFactory. Вы просто создаете новый YAML-файл, описываете в нем роль и инструменты "переводчика", и система автоматически научится его создавать!

Анатомия профиля: заглянем в файл

Профили хранятся в папке `agent_profiles/`. Давайте рассмотрим в качестве примера содержимое файла `researcher.yaml`. Это профиль для нашего агента-исследователя.

```

# agent_profiles/researcher.yaml

# Тип агента: tool_calling может вызывать несколько инструментов одновременно
type: tool_calling

# Описание роли: для чего этот агент нужен?
description: "Ищет информацию в интернете по заданному вопросу, анализирует веб-страницы"

# Модель ИИ: какой "мозг" будет использовать агент
model: model_search

# Инструменты: какими навыками и инструментами он владеет
tools:
  - web_search
  - webpage_content

# Политика памяти: как агент будет использовать свою память
memory_policy:
  # Давать краткую сводку предыдущих действий, чтобы лучше понимать контекст
  provide_run_summary: true
  # Включить семантический поиск по памяти при старте
  search_enabled: true

# Инструкции: базовый промпт, определяющий его поведение
prompt_templates: |
  Ты – элитный исследователь.
  Твоя задача – найти наиболее точную и релевантную информацию по запросу.
  Используй свои инструменты для поиска и анализа.
  Отвечай четко и по существу.

```

Давайте разберем этот файл по частям:

- `type` : Указывает тип базового агента. Например, `tool_calling` лучше справляется с вызовом инструментов.
- `description` : Краткое описание роли агента. `DynamicAgentSystem` использует его, чтобы понять, какой специалист лучше подойдет для задачи.
- `model` : "Мозг" агента. Здесь мы указываем, какую из наших предварительно настроенных моделей ИИ (например, `model_search` , `model_code`) следует использовать.
- `tools` : Список "инструментов", которые будут доступны агенту. В данном случае, это `web_search` для поиска в интернете и `webpage_content` для чтения содержимого страниц.

Подробнее об инструментах мы поговорим в главе [Система загрузки инструментов \(load_tools\)](#).

- `memory_policy` : Настройки памяти агента. Например, `provide_run_summary: true` говорит агенту, что перед началом работы ему нужно освежить в памяти краткое содержание своих прошлых действий. Об этом подробнее в главе [Система памяти RAG \(RagMemory\)](#).
- `prompt_templates` : Самое главное — "должностная инструкция" или системный промпт. Этот текст определяет личность, цели и стиль поведения агента.

Как система загружает и использует профили?

Когда приложение запускается, специальная функция `load_agent_profiles` в файле `agent_command.py` делает очень простую вещь: она сканирует папку `agent_profiles/`, читает каждый `.yaml` файл и складывает все данные в один большой словарь под названием `AGENT_PROFILES`.

```
# agent_command.py

def load_agent_profiles():
    profiles = {}
    profile_dir = 'agent_profiles' # Папка с нашими профилями
    for filename in os.listdir(profile_dir):
        if filename.endswith('.yaml'):
            # Имя файла без .yaml становится ключом (например, 'researcher')
            agent_name = filename[:-5]
            # ...здесь происходит чтение и загрузка YAML-файла...
            profiles[agent_name] = profile_data
    return profiles

# Глобальный словарь, доступный всей системе
AGENT_PROFILES = load_agent_profiles()
```

Теперь, когда `AgentFactory` получает команду создать агента, она просто заглядывает в этот готовый словарь.

```
# agent_factory.py -> метод create_agent()

def create_agent(self, profile_type: str, ...):
    # 'profile_type' здесь – это, например, 'researcher'

    # 1. Загружаем "чертеж" из глобального словаря
    profile = AGENT_PROFILES[profile_type]

    # 2. Используем данные для сборки
    model = profile.get('model')
    tools_list = profile.get('tools', [])
    # ... и так далее
```

Этот механизм элегантен и прост. `AgentFactory` не нужно знать о существовании каких-то конкретных агентов. Ей достаточно получить имя профиля и найти его в словаре `AGENT_PROFILES`.

Давайте создадим нового агента!

Чтобы увидеть всю мощь этого подхода, давайте добавим в нашу систему совершенно нового специалиста — **критика**. Его задача будет состоять в том, чтобы оценивать готовый ответ другого агента и предлагать улучшения.

Шаг 1: Создаем новый YAML-файл

В папке `agent_profiles/` создайте новый файл с именем `critic.yaml`.

Шаг 2: Заполняем профиль

Добавьте в `critic.yaml` следующее содержимое:

```
# agent_profiles/critic.yaml

type: code
description: "Оценивает и критикует предоставленный текст или ответ, чтобы улучшить его"
model: model_lite # Нам не нужна самая мощная модель для этой задачи

tools: [] # Критику не нужны внешние инструменты, он работает только с текстом

memory_policy:
  provide_run_summary: false
  search_enabled: false

prompt_templates: |
  Ты – строгий, но справедливый критик.
  Твоя цель – не просто найти недостатки, а предложить конструктивные улучшения.
  Проанализируй предоставленный тебе текст.
  Выдели сильные стороны, укажи на слабые и предложи конкретные шаги для улучшения.
```

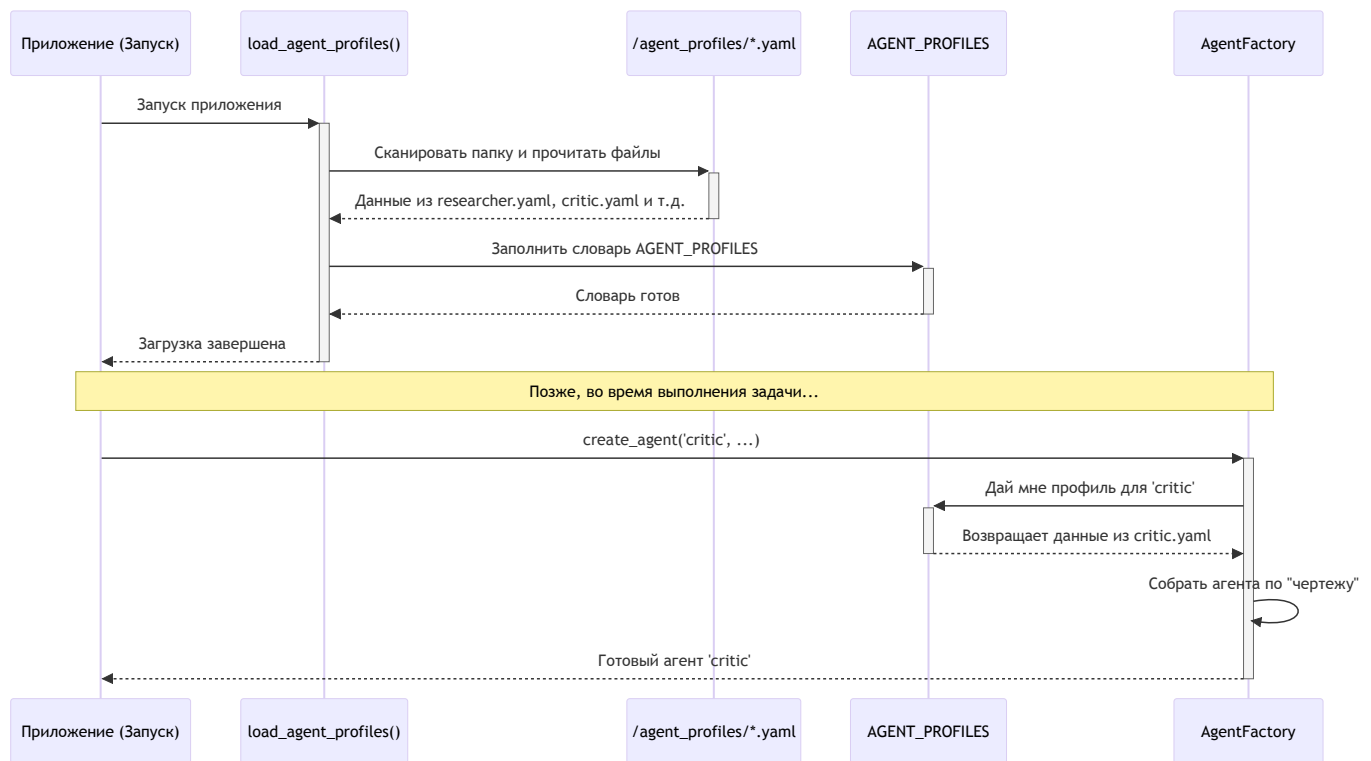
Шаг 3: Готово!

Невероятно, но это всё. Вам не нужно писать ни строчки кода на Python. Просто перезапустите приложение. `load_agent_profiles` автоматически обнаружит новый файл `critic.yaml`, и `AgentFactory` научится создавать агента-критика. Теперь `DynamicAgentSystem` при анализе задачи сможет решить, что для получения высококачественного ответа полезно "нанять" критика для финальной проверки.

Этот пример отлично показывает, как профили отделяют **конфигурацию** агентов от **логики** их создания.

Как это работает "под капотом"

Весь процесс можно представить на простой схеме, которая показывает, как данные из файлов превращаются в работающего агента.



Заключение

В этой главе мы разобрали один из ключевых элементов, обеспечивающих гибкость нашей системы, — профили агентов. Мы узнали, что:

- Профили — это YAML-файлы, которые работают как "должностные инструкции" для агентов.
- Они описывают роль, инструменты, модель ИИ и базовые инструкции для каждого типа агента.
- Система автоматически загружает все профили при старте, создавая глобальный словарь `AGENT_PROFILES`.
- Такой подход позволяет легко добавлять, изменять и настраивать агентов, не трогая основной код программы.

Мы поняли, что такое агент и как он настраивается. Но для эффективной работы агентам нужна способность учиться на своем опыте. Им нужна память.

В следующей главе мы погрузимся в устройство "мозга" наших агентов и рассмотрим, как работает их система долгосрочной памяти. Переходим к изучению [Главы 4: Система памяти RAG \(RagMemory\)](#).

Глава 4: Система памяти RAG (RagMemory)

В [предыдущей главе](#) мы детально разобрали "профили" — YAML-файлы, которые служат чертежами для создания наших агентов. Мы поняли, как описать их роли, инструменты и инструкции. Но даже самый квалифицированный специалист бесполезен, если он не способен учиться на своем опыте.

Представьте команду ученых, которая каждый день начинает свое исследование с чистого листа, забывая все вчерашние открытия, гипотезы и неудачные эксперименты. Такая команда никогда не добьется прорыва. Им нужна общая база знаний, лабораторные журналы и протоколы, чтобы двигаться вперед.

Для наших агентов роль такой коллективной памяти выполняет **RagMemory**.

Что такое RagMemory и зачем она нужна?

Простая память может запомнить последовательность действий: "Шаг 1: поискал в интернете. Шаг 2: проанализировал текст". Но этого недостаточно. Что, если мы хотим спросить: "А что мы уже знаем о методологии Agile из предыдущих задач?". Простая память на такой вопрос не ответит.

RagMemory — это не просто записная книжка, а "умная общая библиотека" и "корпоративная память" для всех агентов. Она построена на технологии **Retrieval-Augmented Generation (RAG)**, что можно перевести как "Генерация, дополненная поиском".

Вот что это значит на практике:

- Хранение (Storage):** RagMemory надежно сохраняет каждое действие, результат и артефакт, созданный агентами, в базе данных.
- Поиск (Retrieval):** Когда появляется новая задача, система не просто смотрит на последние шаги. Она выполняет **семантический поиск** по всей базе знаний. Это значит, что она ищет не по ключевым словам, а по *смыслу*. Запрос "причины отказа от гибких методологий" найдет информацию, даже если в ней использовались слова "проблемы с эджайл" или "недостатки scrum".
- Дополнение (Augmentation):** Найденная релевантная информация (контекст из прошлого опыта) "дополняет" исходную задачу.

4. **Генерация (Generation):** Агент получает не только новую задачу, но и выжимку из релевантного прошлого опыта, что позволяет ему дать более глубокий и качественный ответ, не повторяя старых ошибок.

Проще говоря, RagMemory превращает нашу команду агентов в **обучающуюся систему**, которая с каждой решенной задачей становится "умнее" и опытнее.

Как это настраивается?

К счастью, вся сложность RAG-системы скрыта "под капотом". Нам, как пользователям, не нужно писать сложные запросы к базе данных. Мы управляем поведением памяти через уже знакомые нам [профили агентов](#).

Давайте взглянем на секцию `memory_policy` в файле `researcher.yaml`:

```
# agent_profiles/researcher.yaml

# ... другие настройки ...

memory_policy:
  # Давать краткую сводку предыдущих действий, чтобы лучше понимать контекст
  provide_run_summary: true

  # Включить семантический поиск по памяти при старте
  search_enabled: true
```

Разберем эти два параметра:

- `search_enabled: true` : Этот флаг — ключ ко всей магии. Когда он включен, перед началом работы агент автоматически выполнит семантический поиск по всей истории, чтобы найти релевантный контекст для текущей задачи.
- `provide_run_summary: true` : Эта опция говорит агенту, что в конце своей работы он должен создать краткое саммари (выжимку) о проделанной работе. Эти саммари тоже сохраняются в памяти и в будущем помогают другим агентам (или ему же) быстрее вникнуть в суть прошлых задач.

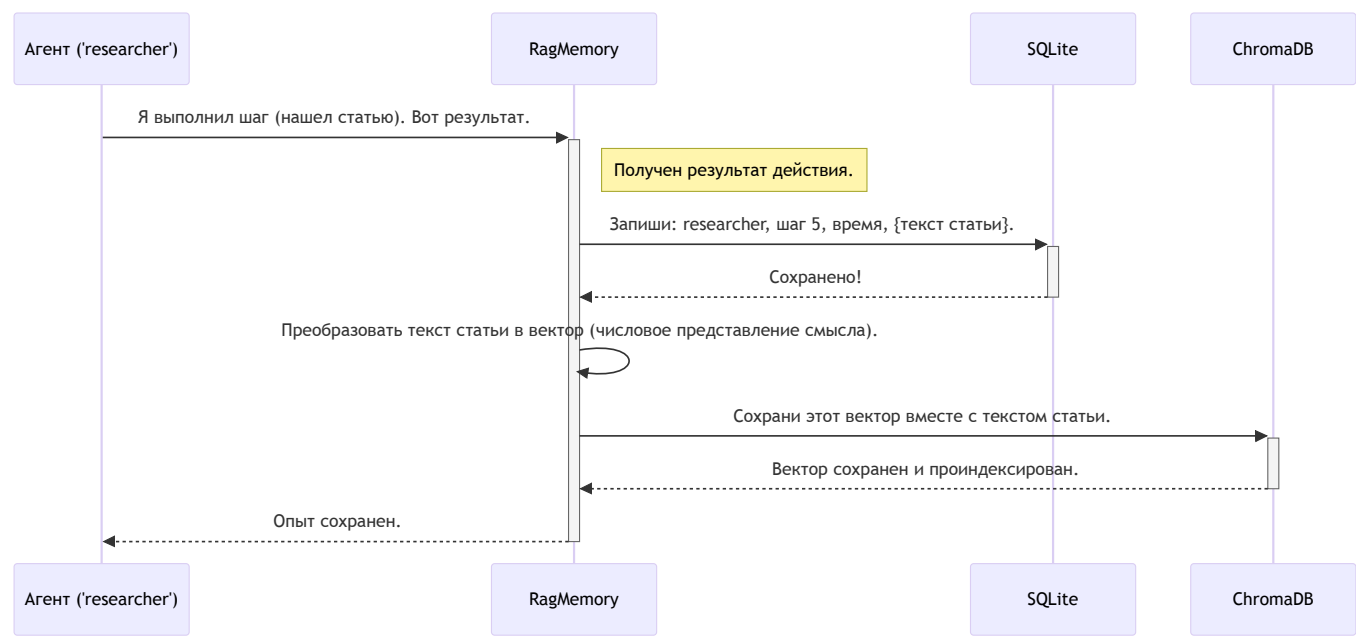
Управляя этими простыми флагами в YAML-файлах, мы можем гибко настраивать, как каждый агент будет использовать свою память.

Как это работает "под капотом"?

Процесс взаимодействия с RagMemory можно разделить на два основных сценария: **сохранение опыта** и **извлечение контекста**. За хранение структурированных данных (кто, что, когда сделал) отвечает база данных **SQLite**, а за "умный" семантический поиск — векторная база данных **ChromaDB**.

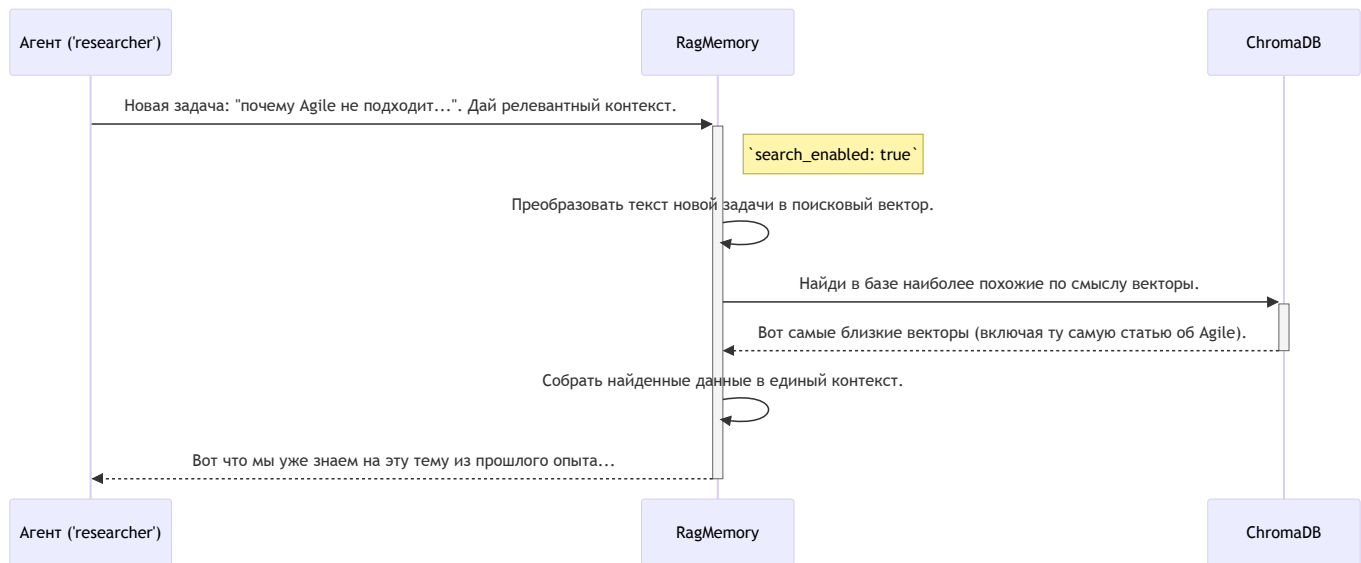
Сценарий 1: Сохранение опыта

Представим, что агент `researcher` только что нашел статью об Agile.



Сценарий 2: Получение релевантного контекста

Теперь представим, что через некоторое время менеджер ставит новую задачу: "Выясни, почему Agile не подходит крупным корпорациям".



Таким образом, агент начинает работу не с нуля, а уже имея на руках полезную информацию.

Логика в коде

Давайте посмотрим на ключевые фрагменты кода, которые реализуют эту логику.

Шаг 1: Создание RagMemory

Когда [Фабрика Агентов \(AgentFactory\)](#) создает нового агента, она также создает для него персональный экземпляр `RagMemory`, считывая настройки из профиля.

```
# agent_factory.py -> метод create_agent()

# Создаем RAG-память (SQLite + ChromaDB) с политикой из профиля
rag_memory = create_rag_memory(
    session_id=session_id,
    agent_name=agent_id,
    profile_config=profile # Передаем конфигурацию для чтения memory_policy
)

# Заменяем стандартную память на нашу RagMemory
agent.memory = rag_memory
```

Функция `create_rag_memory` из файла `memory/rag_memory.py` читает секцию `memory_policy` и настраивает объект памяти с нужными правилами.

Шаг 2: Сохранение шага

Мы не вызываем сохранение напрямую. Система использует "колбэки" — функции, которые автоматически вызываются после каждого действия агента.

```
# agent_factory.py -> метод _build_step_callbacks()

def _save_step(memory_step, agent=None):
    # ...пропускаем подготовку данных...
    payload = {"smol_step_type": ...} # Данные шага

    # Сохраняем шаг в нашу RAG-систему
    agent.memory.add_step(payload)
```

Эта функция передает все данные о шаге в метод `add_step` нашего объекта `RagMemory`.

Внутри `RagMemory` метод `add_step` вызывает функцию `save_memory` (из файла `memory/tools.py`), которая уже распределяет данные: структурированную информацию в SQLite, а семантический вектор — в ChromaDB.

Шаг 3: Получение контекста

Этот процесс тоже автоматизирован. Когда агент готовится выполнить задачу, специальная "обертка" `_wrap_write_memory_to_messages` в `agent_factory.py` перехватывает его обращение к памяти.

```
# agent_factory.py -> метод _wrap_write_memory_to_messages()

def wrapped_write_memory_to_messages(summary_mode: bool = False):
    # ...
    # Проверяем, нужно ли выполнять семантический поиск (search_enabled: true)
    if is_first_step and agent.memory.policy.search_enabled:

        # Выполняем семантический поиск по тексту задачи
        task_search_results = agent.memory.search_memory(
            query=agent.memory.current_run_context
        )
        # ... здесь формируется контекст на основе найденного
    # ...
```

Метод `agent.memory.search_memory` (из `memory/rag_memory.py`) — это и есть тот самый поисковик, который обращается к ChromaDB, чтобы найти релевантную информацию из прошлого опыта.

Заключение

В этой главе мы погрузились в устройство "мозга" нашей мультиагентной системы — RagMemory. Мы узнали, что:

- Это не просто хранилище, а **умная система памяти** на базе RAG.
- Она использует **SQLite** для структурированных данных и **ChromaDB** для семантического поиска по смыслу.
- Это позволяет агентам **учиться на прошлом опыте**, находить релевантную информацию и не повторять ошибок.
- Поведением памяти можно гибко управлять через политику `memory_policy` в **профилях агентов**.
- Вся сложная логика сохранения и поиска **автоматизирована** и не требует ручного вмешательства.

Теперь, когда мы понимаем, как агенты создаются, настраиваются и обучаются, пришло время разобраться с их "руками" — инструментами, с помощью которых они взаимодействуют с внешним миром.

Переходим к изучению [Главы 5: Система загрузки инструментов \(load_tools\)](#).

Глава 5: Система загрузки инструментов (load_tools)

В [предыдущей главе](#) мы разобрались, как агенты запоминают свой опыт с помощью умной памяти RagMemory. Теперь у них есть "мозг" и "память". Но чтобы выполнять реальную работу, им нужны "руки" — инструменты для взаимодействия с внешним миром.

Представьте, что вы наняли самого умного в мире исследователя. Он может анализировать, делать выводы и помнить всё. Но если запереть его в пустой комнате без книг, компьютера и телефона, он будет бесполезен. Ему нужны инструменты. Точно так же нашим агентам нужны инструменты, чтобы искать в интернете, выполнять код или рисовать диаграммы.

За выдачу этих инструментов отвечает **система загрузки инструментов** — наша "служба снабжения".

Что такое система загрузки инструментов?

Это централизованный механизм, который при запуске приложения сканирует все доступные "навыки", собирает их в единый "каталог" и готовит к использованию. Когда [Фабрика Агентов \(AgentFactory\)](#) создает нового агента, она заглядывает в его [профиль](#), видит, какие инструменты ему нужны (например, `web_search`), и берет их из этого общего каталога.

Система умеет "находить" инструменты из двух основных источников:

1. **YAML-файлы:** Простые текстовые файлы в папке `tool_definitions/`, которые описывают инструмент и указывают, где находится его код. Это идеальный способ для добавления собственных, кастомных функций.
2. **МСР-сервисы:** Внешние, готовые к использованию микросервисы, которые предоставляют более сложные возможности (например, работу с базами данных).

Такой подход делает нашу систему невероятно гибкой. Чтобы дать агентам новый навык, нам не нужно переписывать основной код. Достаточно просто описать новый инструмент в YAML-файле!

Анатомия инструмента: смотрим на YAML-определение

Давайте посмотрим, как описан один из самых важных инструментов — `web_search` (поиск в интернете). Это файл `tool_definitions/web_search.yaml`.

```
# tool_definitions/web_search.yaml

# Уникальное имя, по которому мы будем вызывать инструмент
name: web_search

# Откуда брать код для этого инструмента.
# В данном случае, это обычная функция.
source_type: custom_function

# Путь к коду: 'папка.файл.имя_функции'
implementation_source: custom_tools.web_research.web_research
```


Разберем этот файл:

- `name` : Имя инструмента. Именно это имя мы указываем в `tools` в [профиле агента](#).
- `source_type` : Тип источника. `custom_function` означает, что инструмент — это обычная функция в Python.
- `implementation_source` : Путь к функции. Система понимает этот путь так: "в папке `custom_tools` найти файл `web_research.py`, а в нем — функцию `web_research`".

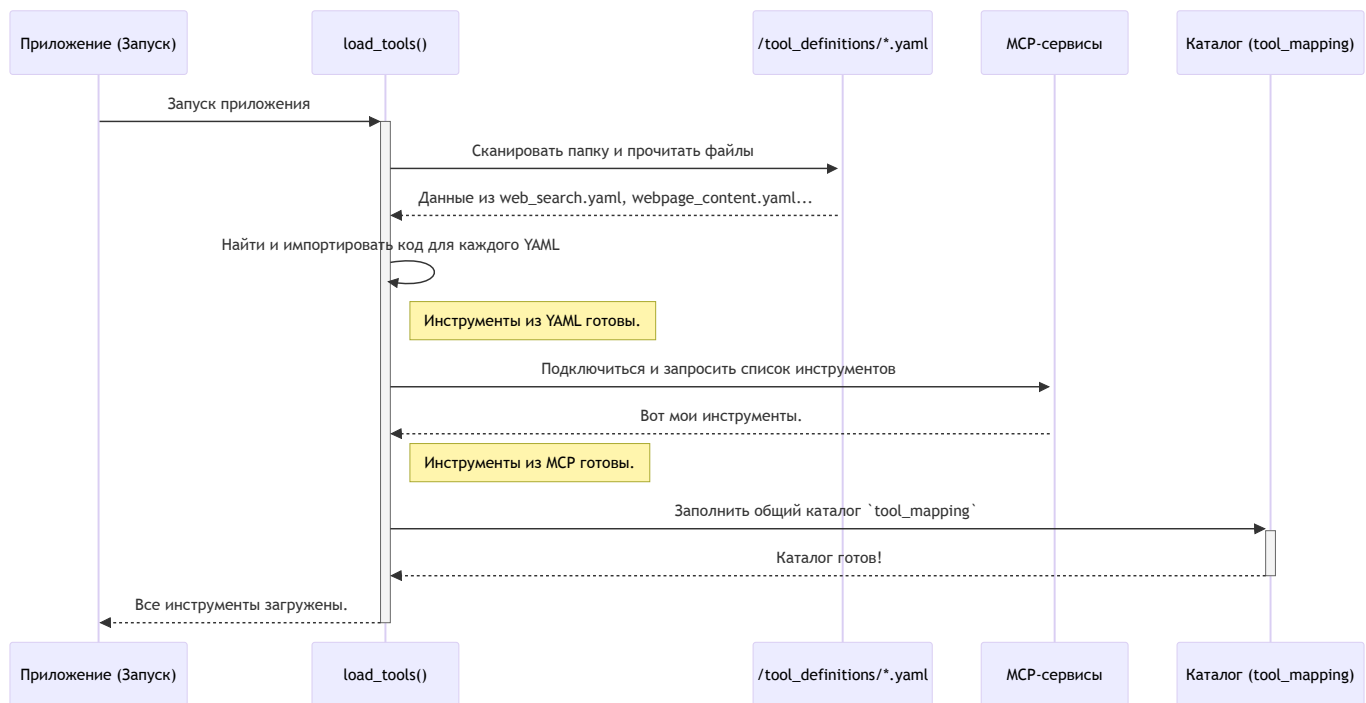
`AgentFactory` будет использовать эту информацию, чтобы найти и "выдать" нужную функцию агенту `researcher`.

Как это работает "под капотом"?

Весь процесс можно разделить на два этапа: подготовка "каталога" при старте и выдача инструмента при создании агента.

Этап 1: Загрузка всех инструментов (`load_tools`)

При запуске приложения вызывается функция `load_tools` из файла `agent_factory.py`. Она работает как кладовщик, который составляет инвентарный список всего, что есть на складе.



Давайте посмотрим на упрощенный код функции `load_tools`.

```
# agent_factory.py -> функция load_tools()

def load_tools():
    tool_mapping = {} # Создаем пустой "каталог"
    tool_dir = 'tool_definitions'

    # Шаг 1: Загрузка из YAML-файлов
    for filename in os.listdir(tool_dir):
        if filename.endswith('.yaml'):
            # ... читаем YAML-файл ...
            tool_config = yaml.safe_load(f)
            tool_name = tool_config['name']
            source_path = tool_config['implementation_source']

            # ... импортируем функцию по пути ...
            module_path, func_name = source_path.rsplit('.', 1)
            module = importlib.import_module(module_path)
            func = getattr(module, func_name)

            # Добавляем инструмент в каталог
            tool_mapping[tool_name] = tool(func)

    # Шаг 2: Интеграция MCP-инструментов (более продвинутый способ)
    for mcp_tool_obj in mcp_tools:
        tool_mapping[mcp_tool_obj.name] = mcp_tool_obj

    return tool_mapping
```

Этот код выполняет две ключевые вещи:

1. Он проходит по всем `.yaml` файлам в папке `tool_definitions`, динамически импортирует код функций, на которые они ссылаются, и складывает их в словарь `tool_mapping`.
2. Затем он добавляет в этот же словарь готовые инструменты, полученные от MCP-сервисов.

В результате `tool_mapping` — это полный и готовый к использованию "каталог" всех навыков в системе.

Этап 2: Выдача инструмента AgentFactory

Этот "каталог" используется внутри [Фабрики Агентов \(AgentFactory\)](#). Когда она создает агента, она вызывает свой внутренний метод `_create_tool`.

```
# agent_factory.py -> AgentFactory

class AgentFactory:
    def __init__(self):
        # При создании фабрики мы сразу же загружаем все инструменты
        self.tool_mapping = load_tools()

    def _create_tool(self, tool_name: str):
        # Просто ищем инструмент по имени в нашем каталоге
        created_tool = self.tool_mapping.get(tool_name)
        if created_tool is None:
            logger.warning(f"ВНИМАНИЕ: Инструмент '{tool_name}' не найден.")
        return created_tool
```

Когда AgentFactory читает профиль researcher и видит в нем `tools: ['web_search']`, она вызывает `self._create_tool('web_search')`. Этот метод мгновенно находит в словаре `self.tool_mapping` нужный объект-инструмент и передает его создаваемому агенту.

Пример: Добавим новый инструмент "Hello World"

Давайте увидим всю мощь этой системы, добавив простейший кастомный инструмент, который просто возвращает строку "Hello, World!".

Шаг 1: Создаем Python-файл с логикой инструмента

В папке `custom_tools/` создайте новый файл `greeting.py`:

```
# custom_tools/greeting.py

def say_hello() -> str:
    """Простой инструмент, который возвращает приветствие."""
    return "Hello, World!"
```

Шаг 2: Создаем YAML-файл с описанием инструмента

В папке `tool_definitions/` создайте файл `greeting.yaml`:

```
# tool_definitions/greeting.yaml

name: say_hello
source_type: custom_function
implementation_source: custom_tools.greeting.say_hello
```

Шаг 3: Добавляем инструмент в профиль агента

Откроем профиль агента, например `researcher.yaml`, и добавим ему новый навык:

```
# agent_profiles/researcher.yaml

# ...
tools:
  - web_search
  - webpage_content
  - say_hello # <-- Наш новый инструмент
# ...
```

Шаг 4: Готово!

Это всё! Больше ничего делать не нужно. При следующем запуске `load_tools` автоматически обнаружит `greeting.yaml`, загрузит функцию `say_hello` и добавит ее в общий каталог.

`AgentFactory` увидит `say_hello` в профиле исследователя и выдаст ему этот инструмент. Теперь наш `researcher` сможет выполнить новую для него задачу: "поприветствуй мир".

Заключение

В этой главе мы познакомились с "сердцем" практических возможностей нашей системы — механизмом загрузки инструментов `load_tools`. Мы узнали, что:

- Инструменты — это "руки" агентов, позволяющие им выполнять реальные действия.
- Система `load_tools` при запуске создает единый "каталог" всех доступных в проекте инструментов.
- Инструменты можно легко добавлять, описывая их в **YAML-файлах** в папке `tool_definitions` и указывая путь к их коду.
- **Фабрика Агентов (AgentFactory)** использует этот каталог, чтобы динамически оснащать агентов нужными навыками в соответствии с их **профилями**.
- Этот подход делает систему невероятно **гибкой и расширяемой**.

Наши агенты теперь полностью оснащены: у них есть инструкции, память и инструменты. Они готовы к выполнению сложных задач. Одна из таких задач — работа с базами данных.

В следующей главе мы рассмотрим, как наши агенты используют специализированный набор инструментов для понимания запросов на естественном языке и преобразования их в SQL-запросы к базам данных. Переходим к изучению [Главы 6: Пайплайн Text-to-SQL](#).

Глава 6: Пайплайн Text-to-SQL

В [предыдущей главе](#) мы узнали, как наша система снабжает агентов "руками" — инструментами для взаимодействия с внешним миром. Теперь у агентов есть всё необходимое для выполнения широкого круга задач. Но что, если задача очень специфична и требует глубоких знаний в определенной области?

Представьте, что вы хотите задать вопрос о данных вашей компании, например: "Покажи мне топ-5 самых продаваемых продуктов в прошлом месяце". Без специальных знаний вам пришлось бы просить аналитика написать сложный SQL-запрос. А что, если бы агент мог сделать это за вас?

Именно для этой цели в MultiAgent существует **Пайплайн Text-to-SQL** — узкоспециализированный "конвейер", который преобразует ваши вопросы на естественном языке в готовые к выполнению SQL-запросы. Это одна из самых мощных и проработанных функций проекта.

Что такое Пайплайн Text-to-SQL?

Пайплайн (или "конвейер") Text-to-SQL — это последовательность шагов, которую выполняет система, чтобы понять ваш вопрос о данных и получить на него ответ из базы данных. Каждый шаг — это отдельный мини-агент или инструмент со своей зоной ответственности.

Представьте себе сборочную линию на заводе:

1. **Начало линии:** Поступает "сырье" — ваш вопрос: "Сколько у нас клиентов из Москвы?".
2. **Первый этап (Понимание):** Рабочий анализирует вопрос и определяет, что нужно "посчитать" (`COUNT`) сущность "клиенты" (`customers`) при условии, что их "город" (`city`) — это "Москва".

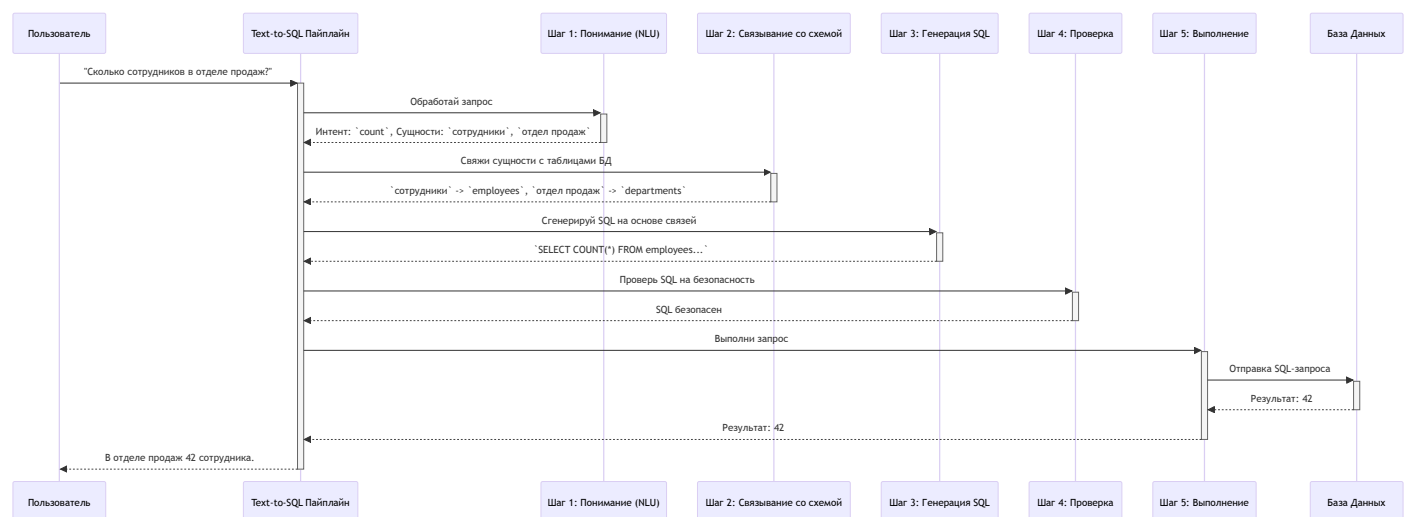
3. **Второй этап (Сопоставление):** Другой рабочий, имея на руках "чертеж" базы данных, находит, что "клиенты" — это таблица `customers`, а "город" — это колонка `city` в этой таблице.
4. **Третий этап (Сборка):** Инженер собирает из этих деталей "деталь" на языке SQL:

```
SELECT COUNT(*) FROM customers WHERE city = 'Москва';
```
5. **Четвертый этап (Контроль качества):** Инспектор проверяет, что в SQL-коде нет опасных команд (вроде `DROP TABLE`), и что он синтаксически корректен.
6. **Пятый этап (Исполнение):** Проверенный SQL-запрос отправляется к станку (базе данных), который его выполняет.
7. **Конец линии:** Вы получаете готовый "продукт" — ответ: "1,250".

Этот конвейер позволяет вам "общаться" с базой данных на человеческом языке, полностью скрывая от вас техническую сложность SQL.

Как работает конвейер?

Весь процесс можно представить в виде следующей диаграммы. Специальный агент, обученный работе с SQL, последовательно вызывает нужные инструменты для каждого этапа.



Что происходит "под капотом"?

Вся логика пайплайна реализована в виде набора инструментов в папке `custom_tools/text_to_sql/`. Главный файл, который собирает их вместе, — `custom_tools/sql_tools.py`. Специализированный `sql-agent` получает эти функции в качестве инструментов и вызывает их в нужной последовательности.

Давайте рассмотрим ключевые функции для каждого этапа.

Шаг 1: Понимание запроса (NLU)

Все начинается с анализа вашего вопроса. Для этого используется функция `natural_language_processing`.

```
# custom_tools/text_to_sql/core.py

def natural_language_processing(text: str) -> Dict[str, List[str]]:
    """LLM-анализ текста: токены и простые POS-теги."""
    # ... использует LLM для разбора предложения ...
    return nlu_processor.process_text(text)
```

Эта функция берет ваш текст ("Сколько сотрудников в отделе продаж?") и с помощью языковой модели (LLM) разбивает его на смысловые части (токены), определяя, что "сколько" — это вопрос о количестве, а "сотрудники" и "отдел продаж" — ключевые сущности.

Шаг 2: Связывание со схемой (Schema Linking)

Это самый сложный и важный этап. Системе нужно понять, как слова из вашего запроса соотносятся с реальной структурой базы данных. Этим занимается функция `schema_linking`.

```
# custom_tools/text_to_sql/core.py

def schema_linking(entities: Dict[str, object]) -> Dict[str, object]:
    """LLM-схемный линкинг с авто-интроспекцией..."""
    # ... сопоставляет "сотрудники" с таблицей `employees` ...
    from .schema_linker import SchemaLinker
    linker = SchemaLinker(...)
    return linker.link_entities_to_schema(entities, {})
```

Она берет сущности ("сотрудники", "отдел продаж") и "смотрит" на схему базы данных, чтобы найти наиболее подходящие таблицы и колонки. Этот процесс использует [Интроспекцию и обогащение схемы БД](#) для получения актуальной информации о структуре базы.

Шаг 3: Генерация SQL

Когда система точно знает, какие таблицы и колонки использовать, она готова сгенерировать SQL-код с помощью `sql_generation_plugin`.

```
# custom_tools/text_to_sql/core.py

def sql_generation_plugin(context: str, user_query: str) -> Dict[str, str]:
    """LLM-генерация SQL из linked_entities..."""
    # ... просит LLM написать SQL на основе контекста ...
    from .sql_generator import SQLGenerator
    generator = SQLGenerator()
    return generator.generate_sql(context, user_query)
```

Эта функция отправляет LLM подробное задание: "Напиши SQL-запрос, который посчитает количество записей в таблице `employees`, соединенной с таблицей `departments`, где имя департамента равно 'Sales'". В результате мы получаем готовый SQL.

Шаг 4: Проверка на безопасность

Прежде чем выполнять какой-либо код, полученный от LLM, его нужно проверить. Функция `sql_safety_check` выступает в роли "службы безопасности".

```
# custom_tools/text_to_sql/core.py

def sql_safety_check(sql_query: str) -> Dict[str, object]:
    """Комбинированная проверка: статическая + LLM-флагование рисков."""
    # ... проверяет SQL на наличие опасных команд ...
    safety_result = sql_validator.validate(sql_query)
    # ...
    return safety_result
```

Внутри она использует `SQLSafetyValidator`, у которого есть черный список опасных ключевых слов.

```
# custom_tools/text_to_sql/validators.py

class SQLSafetyValidator:
    # "Черный список" запрещенных команд
    FORBIDDEN_KEYWORDS = [
        "INSERT", "UPDATE", "DELETE", "DROP", "TRUNCATE", "ALTER", "CREATE",
        "REPLACE", "VACUUM", "ATTACH", "DETACH", "GRANT", "REVOKE"
    ]
    # ... остальная логика проверки ...
```


Если в сгенерированном SQL найдется хотя бы одно из этих слов, выполнение будет немедленно остановлено.

Шаг 5: Безопасное выполнение

Только после успешной проверки SQL-запрос отправляется на выполнение в базу данных через `secure_db_executor`.

```
# custom_tools/text_to_sql/core.py

def secure_db_executor(sql_query: str, ...) -> Dict[str, object]:
    """Безопасное выполнение SELECT... через плагины."""
    # ...
    # Получаем плагин для текущей БД
    plugin = get_plugin(dsn)
    conn = plugin.connect(dsn)
    try:
        # Выполняем SELECT с ограничением по строкам
        return plugin.execute_select(conn, sql_query, row_limit=500)
    finally:
        plugin.close(conn)
```

Эта функция не выполняет запрос напрямую. Она использует [Систему плагинов для баз данных \(DB Plugins\)](#), чтобы безопасно подключиться к базе данных и выполнить запрос. Важно, что она автоматически добавляет лимит на количество возвращаемых строк (например, 500), чтобы случайный запрос не перегрузил систему.

Заключение

В этой главе мы разобрали один из самых сложных и полезных механизмов MultiAgent — пайплайн Text-to-SQL. Мы узнали, что:

- Он представляет собой "конвейер" из нескольких шагов для преобразования вопроса на естественном языке в SQL-запрос.
- Каждый этап (понимание, связывание, генерация, проверка, выполнение) реализован в виде отдельного инструмента.
- Специальный агент вызывает эти инструменты в правильной последовательности для решения задачи.
- Пайплайн включает в себя мощные механизмы безопасности, такие как проверка на запрещенные команды и ограничение на объем возвращаемых данных.

Этот пайплайн — яркий пример того, как множество агентов-специалистов могут работать вместе для решения одной сложной задачи. Но чтобы этот конвейер работал с разными типами баз данных (PostgreSQL, ClickHouse, MySQL), ему нужна универсальность. Как система адаптируется к разным "станкам"?

Об этом мы поговорим в следующей главе, посвященной "адаптерам" и "переходникам" для разных СУБД. Переходим к изучению [Главы 7: Система плагинов для баз данных \(DB Plugins\)](#).

Глава 7: Система плагинов для баз данных (DB Plugins)

В [предыдущей главе](#) мы разобрали сложный, но мощный **Пайплайн Text-to-SQL**, который позволяет нашим агентам "общаться" с базами данных на естественном языке. Мы видели, как он генерирует и выполняет SQL-запросы. Но как он это делает, если в мире существует множество разных баз данных — PostgreSQL, SQLite, MySQL, и у каждой свой "диалект" SQL и свои особенности?

Представьте, что вы путешествуете по миру с ноутбуком. В каждой стране — свой стандарт электрических розеток. Вы же не будете покупать новый блок питания для каждой страны? Конечно, нет. Вы возьмете с собой один универсальный набор адаптеров-переходников.

В нашем проекте роль таких адаптеров выполняет **Система плагинов для баз данных (DB Plugins)**.

Что такое система плагинов для баз данных?

Это набор "универсальных адаптеров", который позволяет основному коду (например, пайплайну Text-to-SQL) работать с любой поддерживаемой базой данных, не зная о её внутренних особенностях.

Каждый плагин — это отдельный модуль, который "знает" всё о конкретной системе управления базами данных (СУБД):

- **Как к ней подключиться?** (используя строку подключения DSN)
- **Как выполнить SQL-запрос?** (с учетом её "диалекта" SQL)
- **Как получить информацию о структуре таблиц (схему)?**

- **Как проверить план выполнения запроса, чтобы оценить его эффективность?**

Благодаря этой системе, наш `sql-agent` остается совершенно "невежественным" по поводу того, с какой именно базой он работает. Он просто просит плагин: "выполни этот запрос" или "дай мне схему", а плагин уже сам решает, как это сделать. Это обеспечивает невероятную **гибкость и расширяемость**. Хотите добавить поддержку новой базы данных? Просто напишите для нее новый плагин-адаптер!

Как это работает?

Вся магия скрыта "под капотом". Основной код не выбирает плагин вручную. За это отвечает специальная функция-менеджер `get_plugin`. Она анализирует **строку подключения (DSN)**, которую вы предоставляете, и автоматически подбирает нужный адаптер.

Например, в коде, который выполняет SQL-запрос:

```
# custom_tools/text_to_sql/core.py -> функция secure_db_executor (упрощенно)
```

```
def secure_db_executor(sql_query: str, dsn: str):  
    """Безопасное выполнение SELECT через плагины."""  
  
    # 1. Автоматически получаем нужный плагин по DSN  
    plugin = get_plugin(dsn)  
  
    # 2. Подключаемся к БД, используя логику плагина  
    conn = plugin.connect(dsn)  
  
    # 3. Выполняем запрос через плагин, не зная, что за БД  
    return plugin.execute_select(conn, sql_query)
```

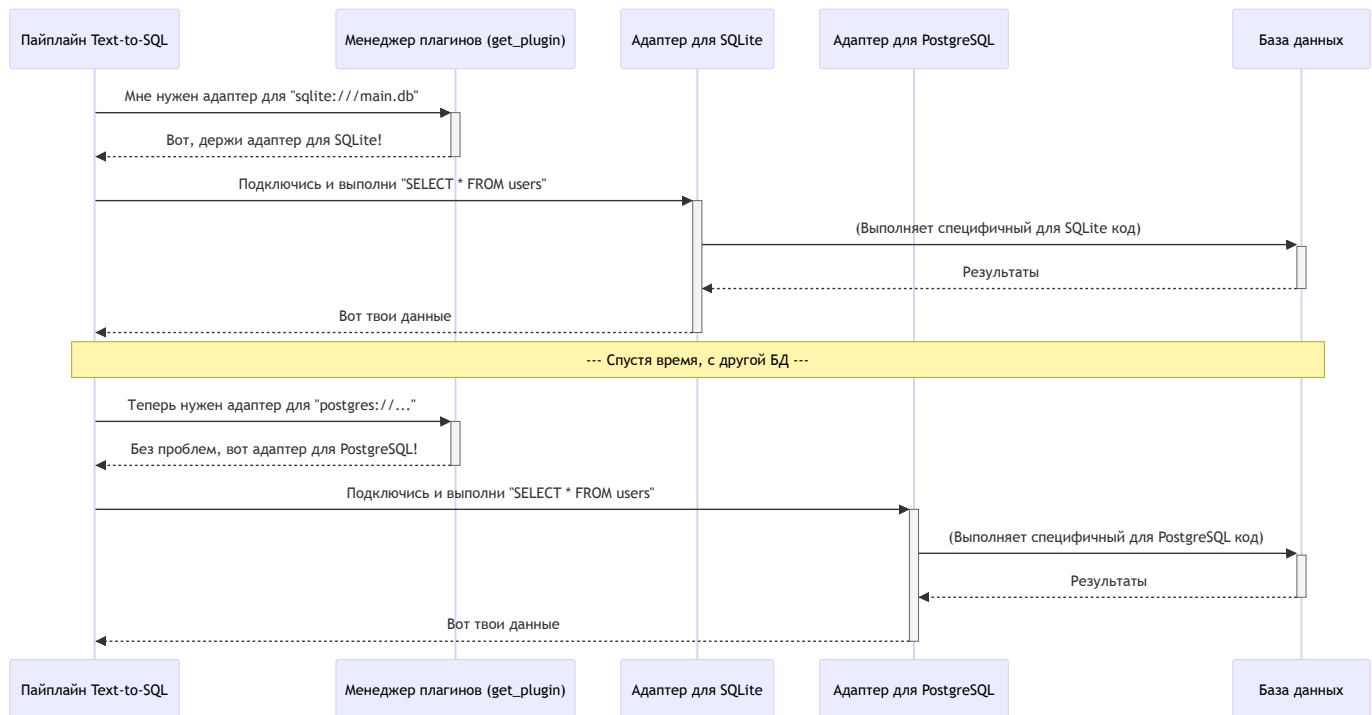
Разберем, что здесь происходит:

1. `plugin = get_plugin(dsn)`: Если `dsn` будет `sqlite:///path/to/my.db`, эта функция вернет экземпляр `SQLitePlugin`. Если `dsn` будет `postgres://user:pass@host/db`, она вернет `PostgresPlugin`.
2. `plugin.connect(dsn)`: Вызывается метод `connect`, но уже у конкретного плагина. `SQLitePlugin` знает, как работать с файлами, а `PostgresPlugin` — как установить сетевое соединение.
3. `plugin.execute_select(...)`: Вызывается метод выполнения запроса. Код остается одинаковым, но `SQLitePlugin` и `PostgresPlugin` выполняют его по-разному, с учетом

особенностей своего "диалекта" SQL.

Что происходит "под капотом"?

Давайте представим эту систему в виде диаграммы. Пайплайн Text-to-SQL — это наш "путешественник". База данных — это "розетка". А система плагинов — это набор адаптеров и тот, кто их выдает.



Как видите, Executor всегда выполняет одни и те же действия, меняется только "адаптер", который он использует.

Шаг 1: "Менеджер плагинов" — get_plugin

Это мозг всей системы. Он находится в файле `db_plugins/manager.py` и представляет собой простой словарь, который сопоставляет название протокола из DSN с нужным классом плагина.

```
# db_plugins/manager.py

_PLUGINS = {
    "sqlite": SQLitePlugin(),
    "postgres": PostgresPlugin(),
    "mysql": MySQLPlugin(),
    # ... и другие плагины
}

def get_plugin(dsn: str):
    # Разбираем DSN, чтобы получить схему (например, 'sqlite')
    parsed = urlparse(dsn)
    scheme = parsed.scheme.lower()

    # Ищем плагин в словаре
    plugin = _PLUGINS.get(scheme)
    if not plugin:
        raise ValueError(f"Нет плагина для схемы: {scheme}")
    return plugin
```

Этот код очень прост: он извлекает `sqlite` из `sqlite:///...` и находит соответствующий объект в словаре `_PLUGINS`.

Шаг 2: "Стандартный разъем" — BaseDBPlugin

Чтобы все "адаптеры" подходили к нашему "устройству" (основному коду), они должны соответствовать единому стандарту. Этот стандарт описан в виде интерфейса (протокола) в файле `db_plugins/base.py`.

```
# db_plugins/base.py (упрощенно)
```

```
class DBPlugin(Protocol):  
    """Интерфейс (стандарт) для всех плагинов БД."""  
  
    def connect(self, dsn: str):  
        """Открывает соединение с БД."""  
        ...  
  
    def execute_select(self, conn, sql: str, row_limit: int = 500):  
        """Выполняет SELECT-запрос."""  
        ...  
  
    def introspect_schema(self, conn, ...):  
        """Получает структуру таблиц и колонок."""  
        ...
```

Этот код не содержит логики. Он лишь объявляет "контракт": любой класс, который хочет считаться плагином, **обязан** иметь эти методы.

Шаг 3: "Конкретный адаптер" — SQLitePlugin

Теперь давайте посмотрим на реализацию одного из "адаптеров", например, для SQLite, в файле `db_plugins/sqlite.py`. Он реализует интерфейс `DBPlugin`, но с логикой, специфичной для SQLite.

```
# db_plugins/sqlite.py

class SQLitePlugin(BaseDBPlugin):
    dialect = "sqlite"

    def connect(self, dsn: str):
        # Логика подключения именно к файлу SQLite
        path = dsn.replace("sqlite://", "", 1)
        conn = sqlite3.connect(f"file:{path}?mode=ro", uri=True)
        return conn

    def execute_select(self, conn, sql: str, row_limit: int = 500):
        # Логика выполнения запроса в SQLite
        # ... добавляем LIMIT, если его нет ...
        cur = conn.cursor()
        cur.execute(sql)
        rows = cur.fetchall()
        # ... возвращаем результат в стандартном формате ...
```

Этот класс содержит конкретный код, который работает только с SQLite. Например, `sqlite3.connect` — это функция из стандартной библиотеки Python для работы с SQLite. Плагин для PostgreSQL будет использовать здесь совершенно другую библиотеку (например, `psycopg2`).

Заключение

В этой главе мы познакомились с элегантной системой плагинов для баз данных, которая является ключом к гибкости и расширяемости нашего проекта. Мы узнали, что:

- Система работает по принципу **универсальных адаптеров**, позволяя основному коду не зависеть от конкретного типа базы данных.
- **Менеджер плагинов** (`get_plugin`) автоматически подбирает нужный "адаптер" на основе строки подключения (DSN).
- Все плагины следуют единому **интерфейсу** (`DBPlugin`), что гарантирует их совместимость с системой.
- Каждый **конкретный плагин** (например, `SQLitePlugin`) инкапсулирует в себе всю логику, специфичную для своей СУБД.
- Такой подход позволяет легко добавлять поддержку новых баз данных, не меняя существующий код.

Теперь, когда наш агент может не только генерировать SQL, но и гибко работать с разными базами, ему осталось научиться делать это максимально "умно". Как агент узнает, какие таблицы и колонки есть в базе данных, чтобы составить правильный запрос?

Об этом мы поговорим в следующей, завершающей главе, посвященной тому, как система "изучает" структуру базы данных. Переходим к изучению [Главы 8: Интроспекция и обогащение схемы БД](#).

Глава 7: Система плагинов для баз данных (DB Plugins)

В [предыдущей главе](#) мы разобрали сложный, но мощный **Пайплайн Text-to-SQL**, который позволяет нашим агентам "общаться" с базами данных на естественном языке. Мы видели, как он генерирует и выполняет SQL-запросы. Но как он это делает, если в мире существует множество разных баз данных — PostgreSQL, SQLite, MySQL, и у каждой свой "диалект" SQL и свои особенности?

Представьте, что вы путешествуете по миру с ноутбуком. В каждой стране — свой стандарт электрических розеток. Вы же не будете покупать новый блок питания для каждой страны? Конечно, нет. Вы возьмете с собой один универсальный набор адаптеров-переходников.

В нашем проекте роль таких адаптеров выполняет **Система плагинов для баз данных (DB Plugins)**.

Что такое система плагинов для баз данных?

Это набор "универсальных адаптеров", который позволяет основному коду (например, пайплайну Text-to-SQL) работать с любой поддерживаемой базой данных, не зная о её внутренних особенностях.

Каждый плагин — это отдельный модуль, который "знает" всё о конкретной системе управления базами данных (СУБД):

- **Как к ней подключиться?** (используя строку подключения DSN)
- **Как выполнить SQL-запрос?** (с учетом её "диалекта" SQL)
- **Как получить информацию о структуре таблиц (схему)?**

- Как проверить план выполнения запроса, чтобы оценить его эффективность?

Благодаря этой системе, наш `sql-agent` остается совершенно "невежественным" по поводу того, с какой именно базой он работает. Он просто просит плагин: "выполни этот запрос" или "дай мне схему", а плагин уже сам решает, как это сделать. Это обеспечивает невероятную **гибкость и расширяемость**. Хотите добавить поддержку новой базы данных? Просто напишите для нее новый плагин-адаптер!

Как это работает?

Вся магия скрыта "под капотом". Основной код не выбирает плагин вручную. За это отвечает специальная функция-менеджер `get_plugin`. Она анализирует **строку подключения (DSN)**, которую вы предоставляете, и автоматически подбирает нужный адаптер.

Например, в коде, который выполняет SQL-запрос:

```
# custom_tools/text_to_sql/core.py -> функция secure_db_executor (упрощенно)
```

```
def secure_db_executor(sql_query: str, dsn: str):  
    """Безопасное выполнение SELECT через плагины."""  
  
    # 1. Автоматически получаем нужный плагин по DSN  
    plugin = get_plugin(dsn)  
  
    # 2. Подключаемся к БД, используя логику плагина  
    conn = plugin.connect(dsn)  
  
    # 3. Выполняем запрос через плагин, не зная, что за БД  
    return plugin.execute_select(conn, sql_query)
```

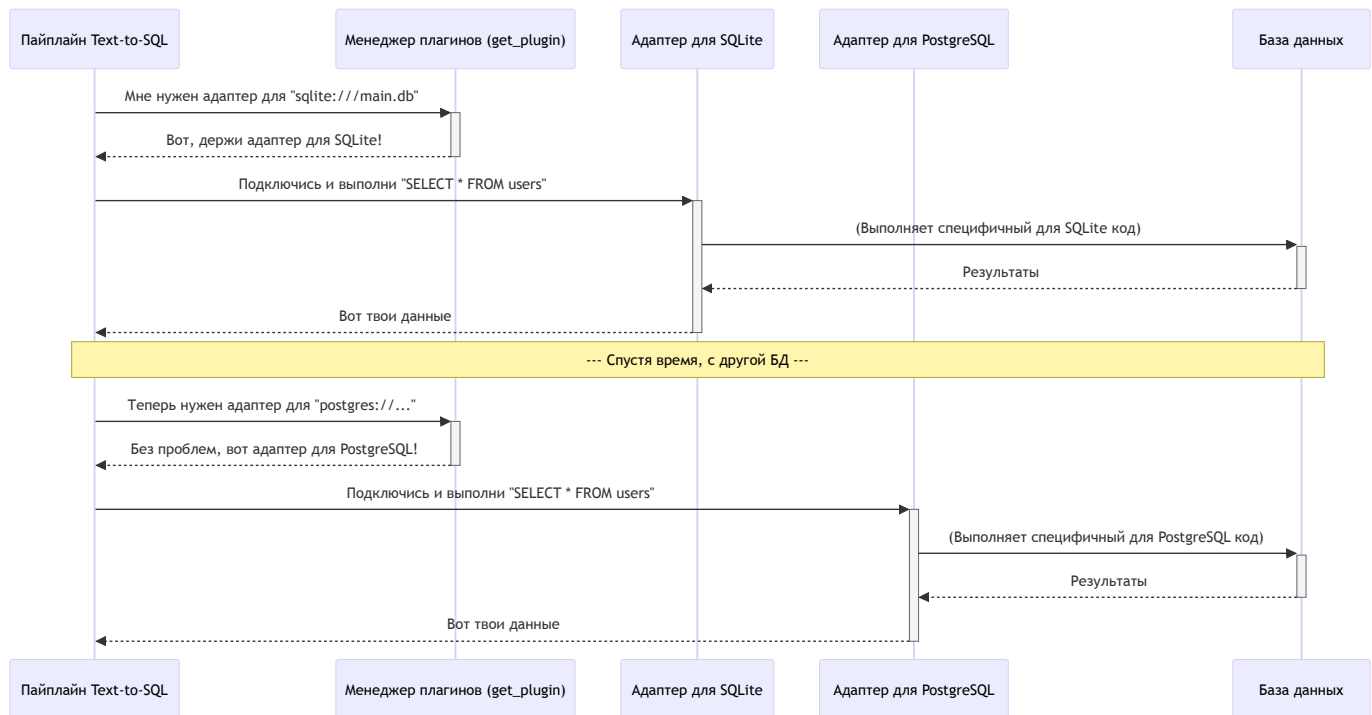
Разберем, что здесь происходит:

1. `plugin = get_plugin(dsn)`: Если `dsn` будет `sqlite:///path/to/my.db`, эта функция вернет экземпляр `SQLitePlugin`. Если `dsn` будет `postgres://user:pass@host/db`, она вернет `PostgresPlugin`.
2. `plugin.connect(dsn)`: Вызывается метод `connect`, но уже у конкретного плагина. `SQLitePlugin` знает, как работать с файлами, а `PostgresPlugin` — как установить сетевое соединение.
3. `plugin.execute_select(...)`: Вызывается метод выполнения запроса. Код остается одинаковым, но `SQLitePlugin` и `PostgresPlugin` выполняют его по-разному, с учетом

особенностей своего "диалекта" SQL.

Что происходит "под капотом"?

Давайте представим эту систему в виде диаграммы. Пайплайн Text-to-SQL — это наш "путешественник". База данных — это "розетка". А система плагинов — это набор адаптеров и тот, кто их выдает.



Как видите, Executor всегда выполняет одни и те же действия, меняется только "адаптер", который он использует.

Шаг 1: "Менеджер плагинов" — get_plugin

Это мозг всей системы. Он находится в файле `db_plugins/manager.py` и представляет собой простой словарь, который сопоставляет название протокола из DSN с нужным классом плагина.

```
# db_plugins/manager.py

_PLUGINS = {
    "sqlite": SQLitePlugin(),
    "postgres": PostgresPlugin(),
    "mysql": MySQLPlugin(),
    # ... и другие плагины
}

def get_plugin(dsn: str):
    # Разбираем DSN, чтобы получить схему (например, 'sqlite')
    parsed = urlparse(dsn)
    scheme = parsed.scheme.lower()

    # Ищем плагин в словаре
    plugin = _PLUGINS.get(scheme)
    if not plugin:
        raise ValueError(f"Нет плагина для схемы: {scheme}")
    return plugin
```

Этот код очень прост: он извлекает `sqlite` из `sqlite:///...` и находит соответствующий объект в словаре `_PLUGINS`.

Шаг 2: "Стандартный разъем" — BaseDBPlugin

Чтобы все "адаптеры" подходили к нашему "устройству" (основному коду), они должны соответствовать единому стандарту. Этот стандарт описан в виде интерфейса (протокола) в файле `db_plugins/base.py`.

```
# db_plugins/base.py (упрощенно)
```

```
class DBPlugin(Protocol):  
    """Интерфейс (стандарт) для всех плагинов БД."""  
  
    def connect(self, dsn: str):  
        """Открывает соединение с БД."""  
        ...  
  
    def execute_select(self, conn, sql: str, row_limit: int = 500):  
        """Выполняет SELECT-запрос."""  
        ...  
  
    def introspect_schema(self, conn, ...):  
        """Получает структуру таблиц и колонок."""  
        ...
```

Этот код не содержит логики. Он лишь объявляет "контракт": любой класс, который хочет считаться плагином, **обязан** иметь эти методы.

Шаг 3: "Конкретный адаптер" — SQLitePlugin

Теперь давайте посмотрим на реализацию одного из "адаптеров", например, для SQLite, в файле `db_plugins/sqlite.py`. Он реализует интерфейс `DBPlugin`, но с логикой, специфичной для SQLite.

```
# db_plugins/sqlite.py

class SQLitePlugin(BaseDBPlugin):
    dialect = "sqlite"

    def connect(self, dsn: str):
        # Логика подключения именно к файлу SQLite
        path = dsn.replace("sqlite://", "", 1)
        conn = sqlite3.connect(f"file:{path}?mode=ro", uri=True)
        return conn

    def execute_select(self, conn, sql: str, row_limit: int = 500):
        # Логика выполнения запроса в SQLite
        # ... добавляем LIMIT, если его нет ...
        cur = conn.cursor()
        cur.execute(sql)
        rows = cur.fetchall()
        # ... возвращаем результат в стандартном формате ...
```

Этот класс содержит конкретный код, который работает только с SQLite. Например, `sqlite3.connect` — это функция из стандартной библиотеки Python для работы с SQLite. Плагин для PostgreSQL будет использовать здесь совершенно другую библиотеку (например, `psycopg2`).

Заключение

В этой главе мы познакомились с элегантной системой плагинов для баз данных, которая является ключом к гибкости и расширяемости нашего проекта. Мы узнали, что:

- Система работает по принципу **универсальных адаптеров**, позволяя основному коду не зависеть от конкретного типа базы данных.
- **Менеджер плагинов** (`get_plugin`) автоматически подбирает нужный "адаптер" на основе строки подключения (DSN).
- Все плагины следуют единому **интерфейсу** (`DBPlugin`), что гарантирует их совместимость с системой.
- Каждый **конкретный плагин** (например, `SQLitePlugin`) инкапсулирует в себе всю логику, специфичную для своей СУБД.
- Такой подход позволяет легко добавлять поддержку новых баз данных, не меняя существующий код.

Теперь, когда наш агент может не только генерировать SQL, но и гибко работать с разными базами, ему осталось научиться делать это максимально "умно". Как агент узнает, какие таблицы и колонки есть в базе данных, чтобы составить правильный запрос?

Об этом мы поговорим в следующей, завершающей главе, посвященной тому, как система "изучает" структуру базы данных. Переходим к изучению [Главы 8: Интроспекция и обогащение схемы БД](#).

Глава 8: Интроспекция и обогащение схемы БД

В [предыдущей главе](#) мы разобрались, как наша система с помощью плагинов-адаптеров может подключаться к самым разным базам данных. Теперь у нашего агента есть "ключи" от хранилища данных. Но что толку от ключей, если он не знает, что лежит на полках?

Представьте, что вас привели на огромный склад и сказали: "Найди деталь для модели T-1000". Без каталога, без маркировки на коробках вы будете блуждать часами. Вам нужен не просто доступ, а **карта склада с пояснениями**.

Именно эту задачу и решает **система интроспекции и обогащения схемы БД**. Это автономный "исследователь данных", который может самостоятельно изучить базу данных и создать для нее ту самую "карту с пояснениями", чтобы агенты могли работать с данными быстро и точно.

Три кита понимания данных

Процесс "картографирования" базы данных состоит из трех ключевых этапов:

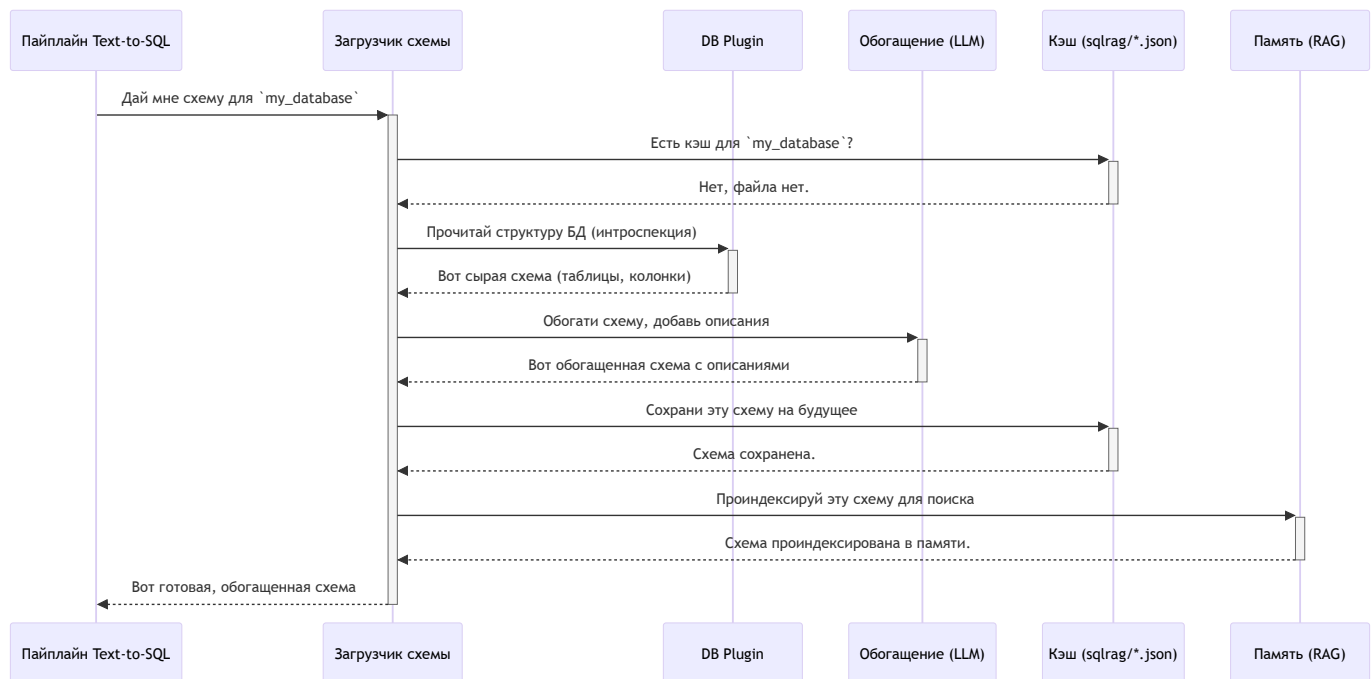
1. **Интроспекция (Introspection):** Это автоматическое "считывание" структуры базы данных. Система, как инспектор, обходит все "комнаты" (таблицы) и переписывает все "коробки" (колонки) и связи между ними (ключи). В результате мы получаем сырую, техническую схему.
2. **Обогащение (Enrichment):** Часто технические названия вроде `prod_rev_fy23` или `usr_auth_tkn` ни о чем не говорят. На этом этапе в игру вступает большая языковая модель (LLM). Она анализирует названия, связи и даже примеры данных, чтобы

сгенерировать человекопонятные описания. Например, для колонки `prod_rev_fy23` она может создать описание: "Выручка от продукта за 2023 финансовый год".

3. **Кэширование и Индексация (Caching & Indexing):** Процессы интроспекции и особенно обогащения могут быть долгими и дорогими. Чтобы не повторять их каждый раз, система сохраняет готовую, обогащенную "карту" в специальный JSON-файл в папке `sqlrag/`. При последующих запусках система будет использовать этот кэш. Более того, эта карта загружается в [систему памяти RAG](#), что позволяет агентам выполнять по ней семантический поиск.

Как это работает?

Весь этот сложный процесс запускается автоматически, когда агент, работающий с [Пайплайном Text-to-SQL](#), впервые обращается к незнакомой базе данных.



Как только этот одноразовый процесс завершен, при всех последующих запросах к той же базе данных система мгновенно загрузит готовую схему из кэша.

Что происходит "под капотом"?

Давайте заглянем в код и посмотрим на трех главных действующих лиц этого процесса: `SchemaLoader`, `SchemaEnricher` и `SchemaMemoryManager`.

Шаг 1: Загрузка схемы (SchemaLoader)

Все начинается в файле `custom_tools/text_to_sql/schema_loader.py` . Класс `SchemaLoader` — это диспетчер, который решает, откуда брать схему.

Его главный метод `get_database_schema` работает по простому алгоритму: сначала ищет кэш, и только если его нет — запускает "исследование".

```
// custom_tools/text_to_sql/schema_loader.py -> SchemaLoader

def get_database_schema(self, ...):
    # 1. Пытаемся загрузить схему из кэш-файла
    dsn = os.getenv("DB_DSN")
    sqlrag_schema = self._load_sqlrag_schema(dsn)

    if sqlrag_schema:
        # Если кэш есть – используем его
        logger.info(f"✅ Схема загружена из файла-кэша...")
        return sqlrag_schema

    # 2. Если кэша нет, "читаем" схему напрямую из БД
    logger.info("Кэш не найден, запускаем интроспекцию БД...")
    return self._introspect_via_plugin(dsn)
```

Функция `_introspect_via_plugin` использует [DB плагины](#) для выполнения сырой интроспекции.

Шаг 2: Обогащение схемы (SchemaEnricher)

Если `SchemaLoader` получил "сырую" схему без описаний, он передает ее "художнику-оформителю" — `SchemaEnricher` из файла `custom_tools/text_to_sql/schema_enricher.py` .

Этот класс находит все таблицы и колонки без описаний и отправляет их в LLM для генерации этих описаний.


```
// custom_tools/text_to_sql/schema_enricher.py -> SchemaEnricher

def enrich_descriptions_with_llm(self, schema_obj):
    # 1. Находим все элементы, у которых нет описаний
    tables_to_process = self._find_undescribed_items(schema_obj)

    if not tables_to_process:
        return # Все уже описано, работа не нужна

    # 2. Для каждой таблицы...
    for table_name in tables_to_process:
        # ... получаем примеры реальных данных для лучшего контекста
        sample_data = self.get_table_sample_data(table_name)

        # ... формируем умный промпт с контекстом и примерами
        prompt = build_column_description_prompt_with_context(...)

        # ... просим LLM сгенерировать описания
        response = call_openai_api(prompt, ...)

        # ... здесь парсинг ответа и обновление объекта схемы ...
```

Ключевой момент здесь — это `get_table_sample_data`. LLM дает гораздо более точные описания, если видит не только названия `id`, `created_at`, `user_name`, но и реальные данные: `1, '2023-10-26', 'John Doe'`.

После обогащения `SchemaLoader` сохраняет результат в JSON-файл в папке `sqlrag/` для будущих поколений.

Шаг 3: Индексация в памяти (`SchemaMemoryManager`)

Финальный и очень важный этап — загрузка обогащенной схемы в "оперативную память" системы, чтобы агенты могли по ней быстро искать. Этим занимается `SchemaMemoryManager` из `custom_tools/text_to_sql/schema_memory.py`.

```
// custom_tools/text_to_sql/schema_memory.py -> SchemaMemoryManager

def index_schema_in_memory(self, session_id, db_schema, ...):
    # Для каждой таблицы в нашей схеме...
    for table_fqn, table_schema in db_schema.items():

        # 1. Создаем из схемы одно большое текстовое описание
        table_description_for_search = self.create_table_description(...)
        # Пример: "Таблица public.users. Описание: Данные о пользователях.
        # Важные колонки: user_id, name, email."

        metadata = { "description": table_description_for_search, ... }

        # 2. Сохраняем это описание в RAG-память
        save_memory(
            session_id=session_id,
            agent_name="Schema-RAG-Agent",
            data=metadata
        )
```

После этого агент, получив задачу "покажи почту всех пользователей", может выполнить семантический поиск по своей [RAG-памяти](#). Память ему подскажет, что слова "почта" и "пользователи" очень близки по смыслу к описанию таблицы `public.users`, и тем самым направит пайплайн Text-to-SQL по верному пути.

В том же файле находится и логика поиска. Функция `find_semantic_relevant_tables` берет сущности из запроса пользователя ("почта", "пользователи") и ищет наиболее похожие описания таблиц в памяти.

```
// custom_tools/text_to_sql/schema_memory.py -> SchemaMemoryManager

def find_semantic_relevant_tables(self, entities: List[str]) -> List[str]:
    # ... подготовка запроса ...
    search_query = " ".join(entities)

    # Выполняем семантический поиск по памяти RAG
    semantic_search_results = memory_manager._search_semantic_with_scores(
        ...
        search_query,
        where={"cache_kind": "schema_table"} # Ищем только по схемам
    )

    # ... фильтруем результаты по релевантности и возвращаем имена таблиц ...
    return relevant_tables
```

Заключение

В этой главе мы увидели, как система `MultiAgent` превращает "незнакомую территорию" базы данных в подробную, аннотированную и удобную для поиска карту. Мы узнали, что:

- Процесс состоит из **интроспекции**, **обогащения** с помощью LLM и **кэширования**.
- **SchemaLoader** выступает в роли диспетчера, решая, загрузить схему из кэша или запустить полное исследование.
- **SchemaEnricher** использует LLM, чтобы добавить человекопонятные описания к техническим названиям, значительно повышая точность агентов.
- **SchemaMemoryManager** индексирует готовую схему в **памяти RAG**, делая ее доступной для быстрого семантического поиска.

Это был заключительный фрагмент мозаики. Теперь у вас есть полное представление обо всех ключевых механизмах системы `MultiAgent` — от высокоуровневой координации в `DynamicAgentSystem` до глубокой, автономной работы с данными. Вооружившись этими знаниями, вы готовы не только использовать, но и расширять возможности этого мощного инструмента.